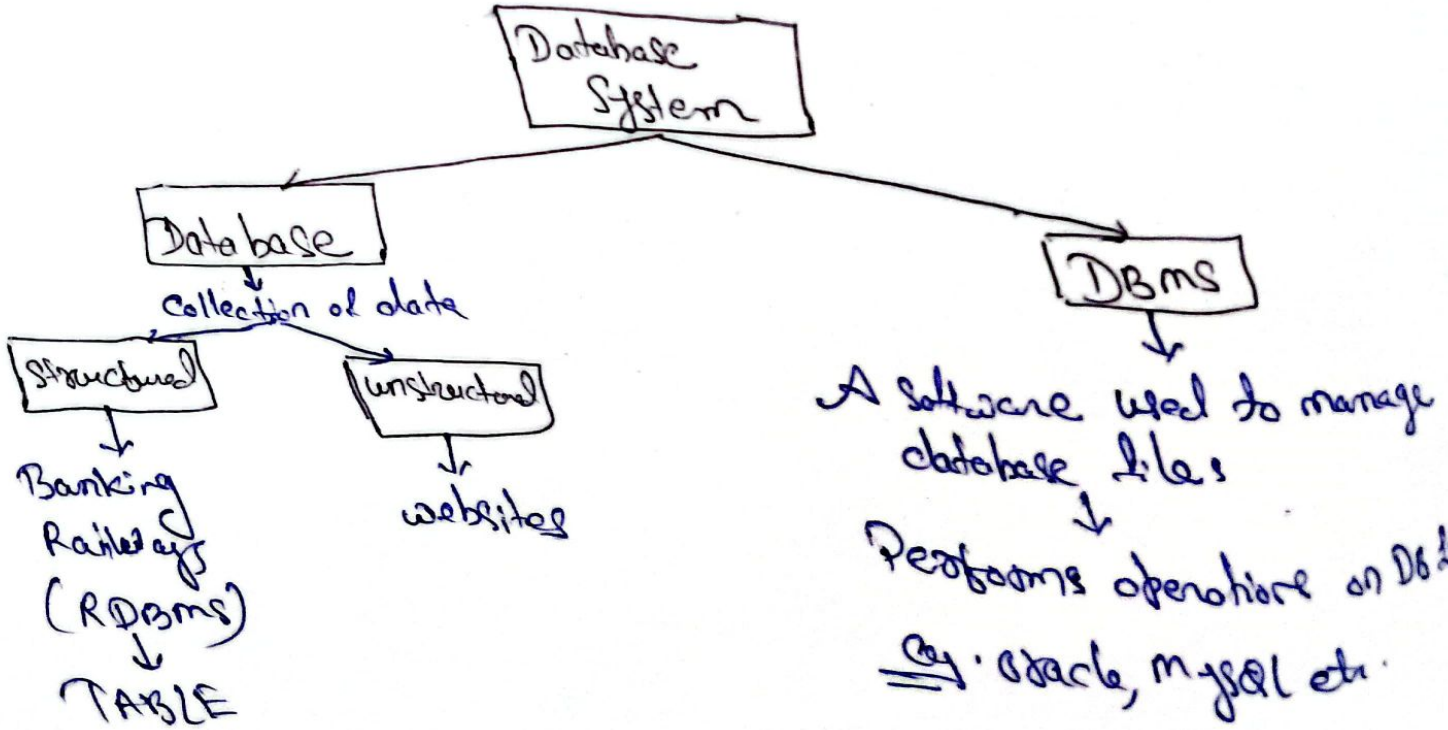
DBMS

(1) Data :- Raw facts or figures which are in an organized form and do not have any proper meaning are called data.
Eg: Text, Number, audio ... etc.

(2) Information :- When various operations are performed on data and it becomes meaningful/processed. Ex: table.

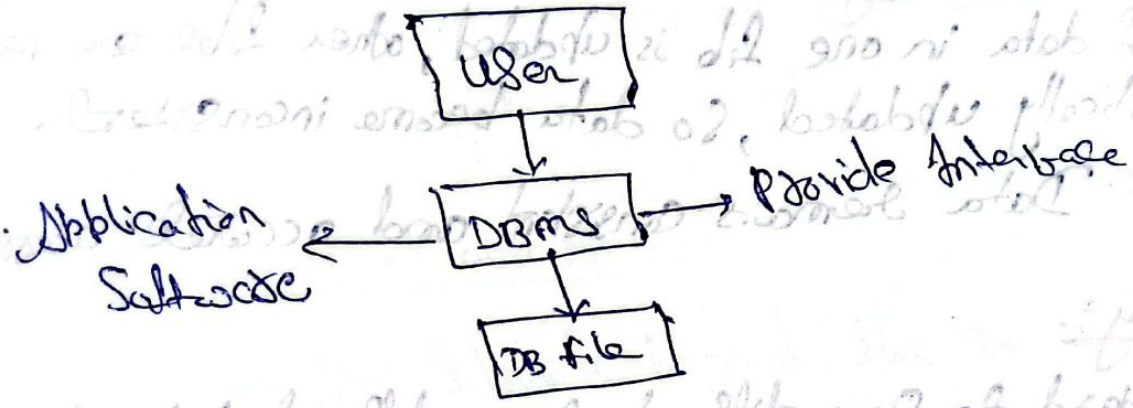
Data → Process → Information → Knowledge → Wisdom.

* Database :-
→ Connection of inter related data.
Types :- Structured & unstructured.



* DBMS:-

→ It acts as an interface b/w the user and the DB file.
→ It converts the user's req into database Query, Sends it to the DB file, fetches the required data, and return the information to the user.
→ It is a collection of application software that enables communication b/w the user and the DB file.



* File System:-

→ To manage various files in computer.
→ managed by OS.
→ 2 Types:-
  - FAT (File Allocation Table) - Basic
  - NTFS (New Technology file System)
    - main file (master file System)
    - made for Window NT kernel (1993)

→ use for small files.

* Difference between file system and DBMS:-

(1) Data Redundancy:-

- File System:- Same data gets repeated/duplicated in many files.
- DBMS:- No duplication.

(2) Data Inconsistency:-

- fs:- If data in one file is updated, other files are not automatically updated, so data become inconsistent.
- DBMS:- Data remains consistent and accurate access.

(3) Security:-

- fs:- Hard to give different users different type of access.
- DBMS:- Advanced security, can give multiple users different access permission.

(4) Concurrency:-

- fs:- Multiple users cannot reliable access the same file at the same time.
- DBMS:- Multiple users can access same data/file at the same time safely.

* In DBMS:- A user can access data without knowing all internal details. The user is given only the information needed. (Specific row, table, col^n)

* Data access cost (file system / DBMS):-

→ In fs, bringing data from sec mem to primary memory is more costly because the entire file must be stored.

→ In DBMS, Indexing is used, low cost

* Concurrency level :-

(a) File System :- Concurrency is at the file/resource level. When one user is using a file, another user cannot use (access/modify) the same file at the same time.

(b) DBMS :- Concurrency is at the row/record level. When one user is using a particular row, only that row is locked for other user, all other rows remains available for everyone.

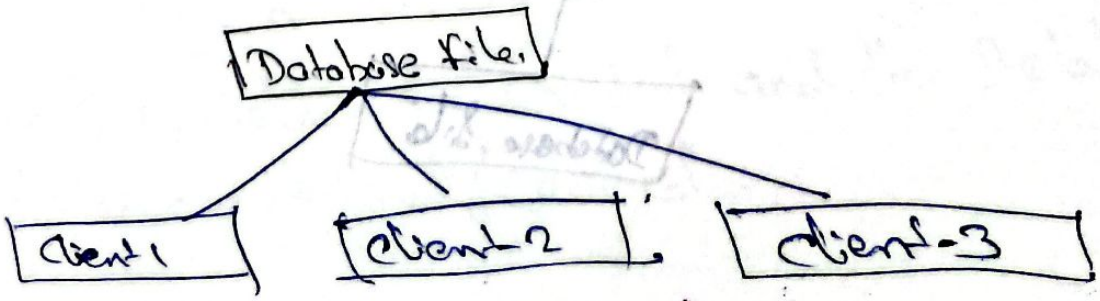
* Different level of access :-

- File System :- No fine, seperate physical file are needed.
- DBMS :- Different view created.

VIEW = Virtual Table :-

→ It performs operations on the underlying master table
→ It does not physically store data.
→ It only looks like the original physical table.

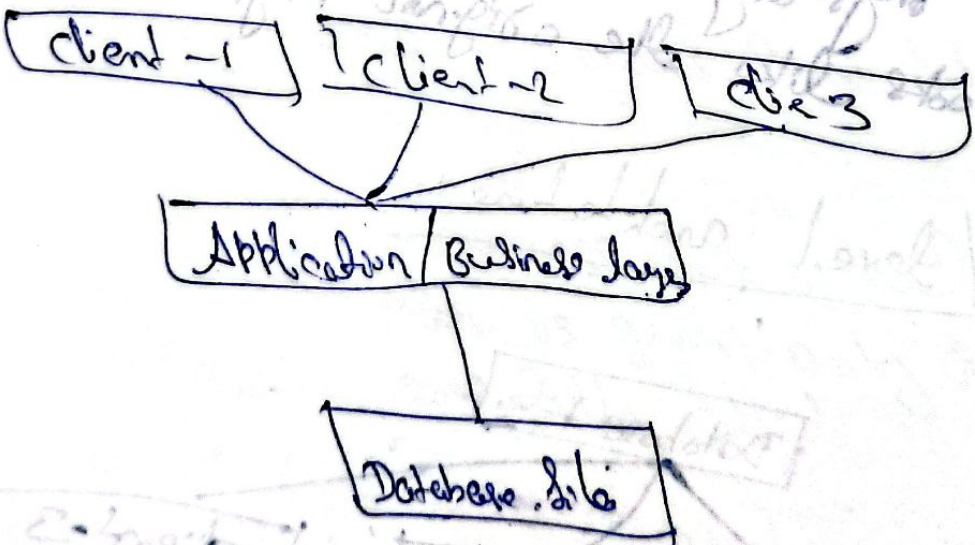
* Two tier / level architecture :-



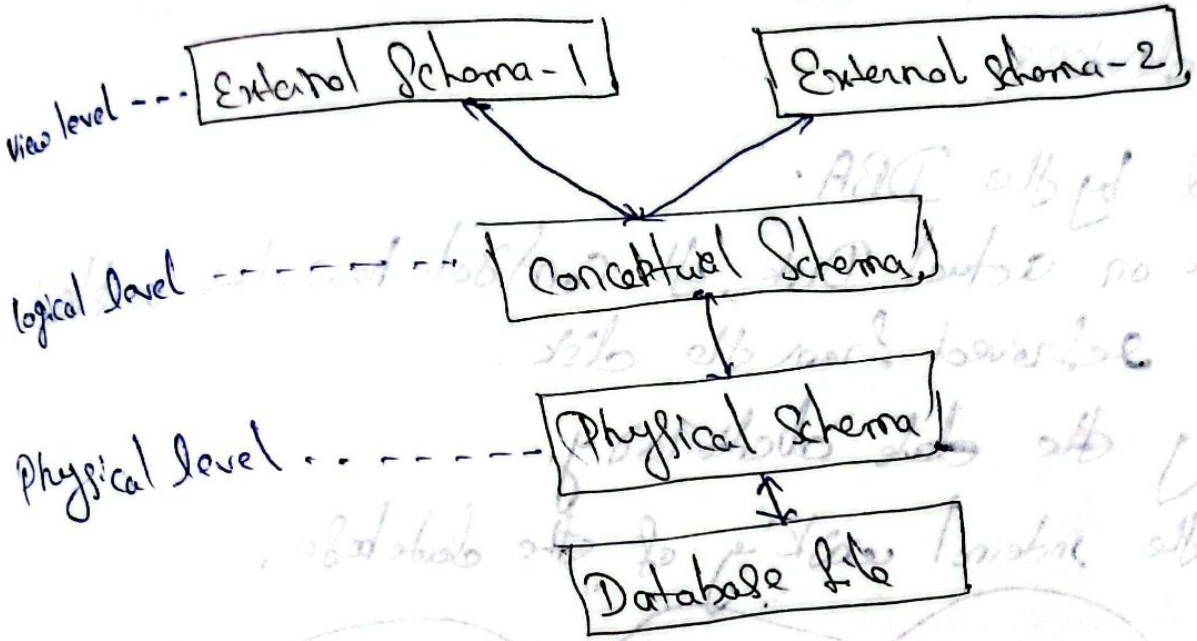
→ The client connects directly to the database server.
→ client-server architecture
→ limited no. of users
→ less secure, easy to maintain.
→ The process on the client is converted into database query (often called a low-level language query) and sent to server.

* 3 tier / 3 level architecture.

→ The user/client doesn't directly connected to the DB server
→ The client's req. first sent to the application / Bussiness layer
→ The application layer then converts the request to a low level database query and interacts with DB server
→ The application layer acts as an interface b/w client & server
→ Because clients do not talk to the server directly, its architecture provide better security.
→ most web apps (eg: fb, insta, amazon)



* 3 level Abstraction / 3 Schema Abstraction :-



(1) External Schema / view level :-

→ Users are connected to the DB.
→ Each user gets a different view/Schema acc to need.
→ Conceptual and physical details are hidden from user.
→ which table actually stores the user's data, how the table is related to other tables and on which disk the data is stored — all hidden.

(2) Conceptual Schema / Logical level :-

→ Describes what data is stored in the database.
→ overall design of the DB is made using this Schema.
→ Based on this, tables are created and the relationships b/w tables are defined / created.

(3) Physical Schema/Internal Schema:-

→ Shows how the actual database files are stored on storage devices.

→ Managed by the DBA.

→ We work on actual Disk, It controls how data is stored on and retrieved from the disk.

→ maintaining the data dictionary

→ Controls the internal working of the database.

Physical Schema = Deals in Storage of DB
Conceptual Schema = Deals in table & their relationship
External Schema = Deals in Requirement of user.

* Data Dependence.

(A) logical data dependency:-

→ If we add a new column to a table or change relationships between tables, the end-user interface (view) doesn't change

→ View level ⟷ Conceptual level. (Independence b/w)

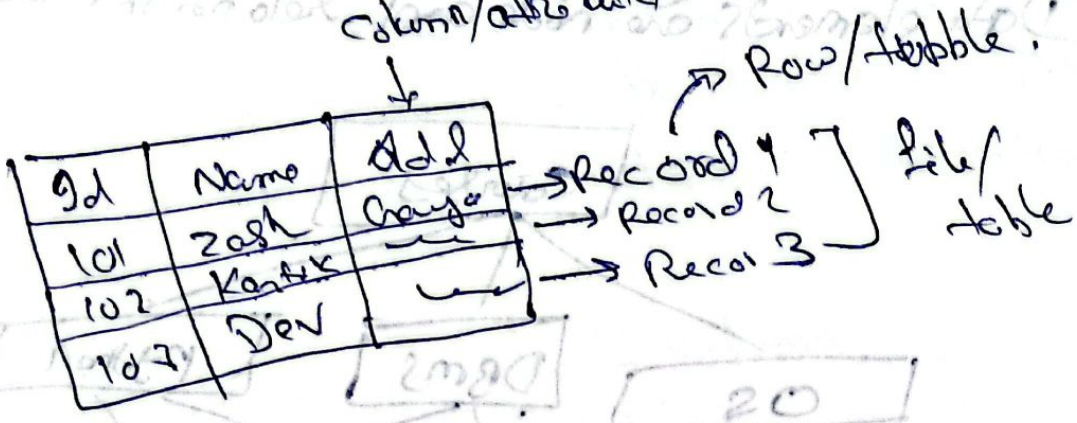
(B) Physical data dependency:-

→ If we add new storage/disk or change the diff location of data, the table and their relationship are not affected.

→ Independence b/w conceptual level ⟷ Physical level (Strong)

* Some points :-

1) Bits - Smallest unit to represent data.
2) character - collection of Bit.
3) Fields - collection of character.
4) Record - collect of field
5) file - collection of Record
6) Database - collection of fields.

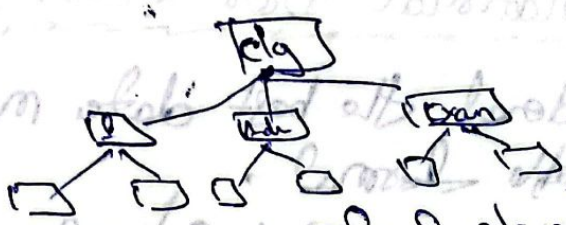


* Data Model :-

→ Data representation in structural way.

Types:

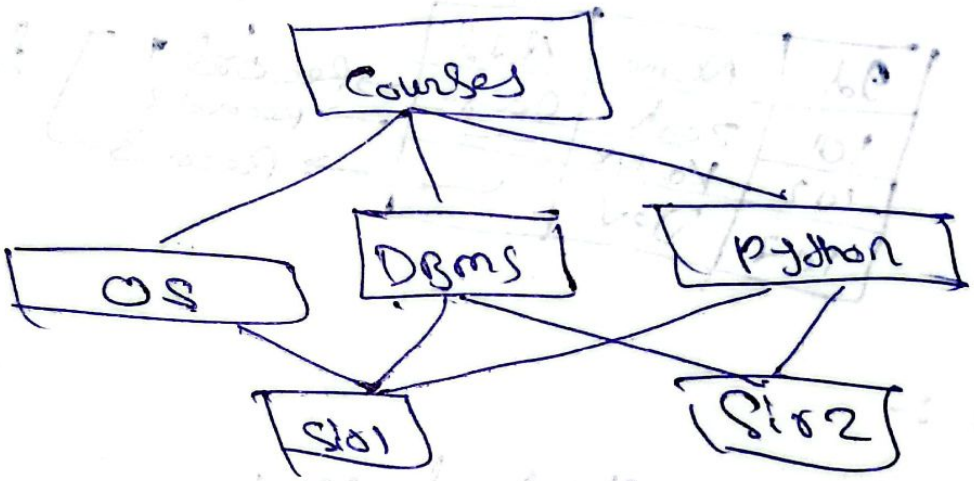
(1) Hierarchical Data model :-



→ Developed by IBM (1968)
→ Developed for IMS (Information management system)
→ Represents parent-child / one to many relationship.
→ Tree data model
→ In this model, one child of a parent cannot see another Child's information.
→ Single-parent model
→ Each record is called a node / segment.
→ top-down approach.

(2) Network Data Model :-

→ Extention of the Hierarchical Data model.
→ Based on Parent-child relationship.
→ one child - can have one parent.
→ Uses graph data structure.
→ many to many relationships.
→ Data elements are node and relationship are edges in path.



(3) Relational Data Model :-

→ considered the best data model because data is represented in table format.
→ A table = row + column
→ A table is called a relation.
→ column represents property of one entity.
→ Each record in the table should not be repeated.
→ A row (record/table) represent an entity.
→ This model follows E.f codd's first rule: data must be arranged in the form of rows & column in table.
→ E.f codd defined 12 rules for the relational model.

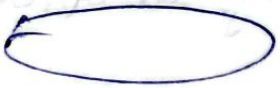
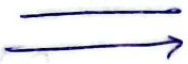
(a) Entity-Relationship (ER) model :-

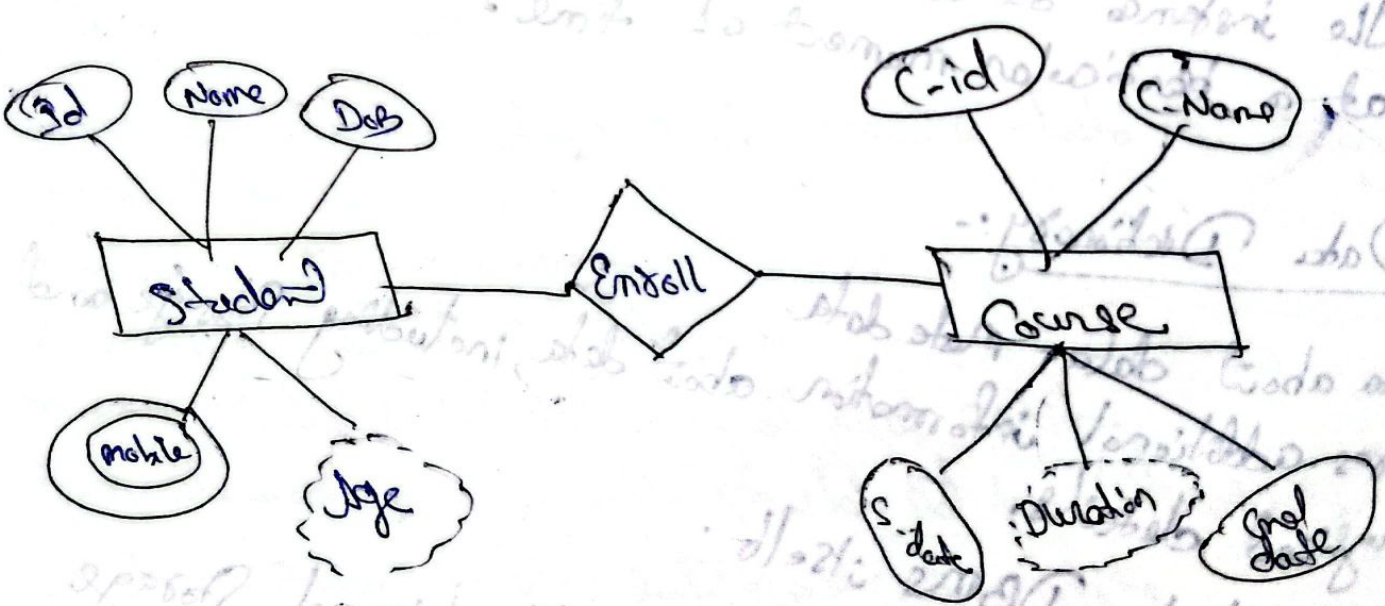
(A) Entity
→ Any object that has a physical apperance or exists physically is called an entity.
ex. for Student table, attributes like, Rollno, name describe the Student entity set. Each individual Student is an entity.
→ In this data model, the database is represented graphically, so the complete logical structure of the database is shown.
→ Using this model, the schema of the database is represented theoritically.
→ The logical representation of data — Schema
→ Student (Id, name, Class, Phone) — Relation
→ A Schema shows things like table, Primary key, foreign key.
→ The instance of DB is the snapshot of the data available at a particular moment of time.

* Data Dictionary :-
→ Data about data - meta data
→ Stores additional information about data, including storage and physical details.
→ managed by DBMS itself.
→ Stored in the DB's tablespace (the internal storage area where all the tables are stored).

(B) Relation → Connection of 2 entity.
(C) Attributes → Characteristic of entity.

* Symbols:

~~Rectangle~~ Entity.		Entity
Ellipse ~~Attributes~~.		attribute
Diamonds		Relationship
Lines.		Entity of relationship Entity of Attributes
Double Ellipse.		Multivalued attribute
Dashed ellipse		Derived attribute
Double Rectangle		weak Entity
Double diamond		weak entity relationship
Ellipse with line		key attribute



* Types of Attributes :-

1) Single value → Carry Single value - Roll no.
2) multivalue → Carry multi value - mobile no.
3) Simble → Registration no. (carry Single value)
4) Composite → Name (f. Name, Last name)
5) Stored → Attribute given to the table.
6) Derived → Is not part of table (Age)
7) Key → To identify record, uniquely (key ✱ primary)
8) Non-key → Not uniquely identify the record (Name, DOB)
9) Complex → multivalued + composite. (Address)
10) Required → compulsory attribute (Not Null)
11) optional → Not compulsory.

---

12) object oriented → Stored data in form of object.
    - value store in objects instance variable
    - use in high LL.
    - Support Complex Datatype & Inheritance.
13) Document Data model.
    → Subset of other Data model, No Sql use.
    → use Json, to make the object in programming.
14) Key-value Data model :-
    → No Sql use
    → Simple & Fast.

* Keys in DBMS:-

→ Uniquely identifies each record/tuple.
→ According to E.f could rule 1. Two records in a table cannot be identical.
→ A key can be formed from one or more attributes.

* Types:

(1) Candidate key:-
→ The minimum set of attributes that can uniquely identify a table/record is called Candidate key.
→ Column Sid, address, Name, if Sid uniquely identify each record, then S_id is a candidate key.
→ A table can have multiple unique key; one is chosen as a primary key, and the remaining are still Candidate keys.
→ values of a candidate key must always be unique and NOT NULL.
→ A candidate key can be simple or composite.
(A, BC, E) → A, E BC

(2) Primary key:-
→ must be unique and not null.
→ Primary key is also a candidate key whose values always unique & not null.

Student (Sid, Name, class, Phone)
Sid → Primary
Phone → Alternate key
Candidate key.

→ A table can have only one primary key.

→ Alternate key:-

→ All candidate key except the chosen primary key.

→ may be allowed to be null.

→ also called Secondary key

* Prime/key attributes :-

Relation R (A, B, C, D, E)

↓

Candidate key (A, BC, E)

↓

Primary Attribute (A, B, C, E)

Then D become,

Non primary key attribute.

(1) Super key:-

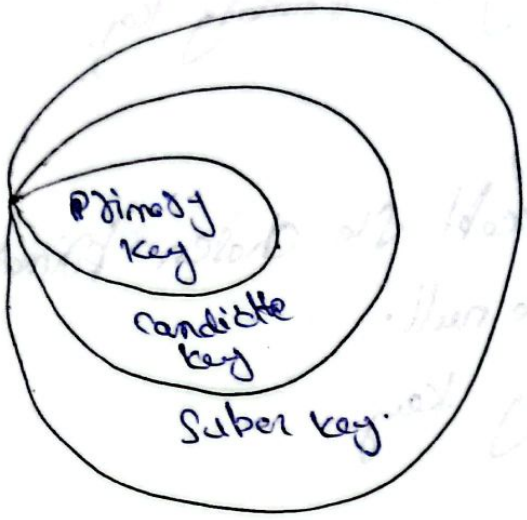
→ A set of attribute that can uniquely identify a table/record in a table.

→ (Sid, Name), (Name, F_Name), (Sid, Class) etc ← Super key

→ Every candidate key is a super key but not every super key is a candidate key (because it may contain extra, unnecessary attributes.

→ must contain atleast one attribute of a candidate key, but it is not necessary that all its attributes are from the candidate key.

→ The minimum super key are called candidate key.



** Master Capsule :-

1) If candidate is one (A) = Number of Super key.
2^{n-1}, n = No. of attributes in a relation

2) If candidate key is Two (A, B) = Number of Super key.
$2^{n-1} + 2^{n-1} - 2^{n-2}$.

3) If candidate is AB = Number of Super key.
2^{n-2}.

Que 1. Relation $R_1(A, B, C, D)$, & CK = 'A'.

A = No. Super Key = $2^{n-1} = 2^{4-1} = 2^3 = 8$.

Q 2: $R_1(A, B, C, D)$, CK = 'A' & 'B'.

A. no. of Superk = $2^{n-1} + 2^{n-1} - 2^{n-2} = 2^3 + 2^3 - 2^2 = 12$.

Que → $R_1(A, B, C, D)$, and CK = ABCD.

A → no. of Super key = [1].

5) Foreign / Reference key :-

→ Primary key of one table is used as column in some other table,
→ Can be Null and not unique.
→ No uniqueness constraint apply.
→ Reference table contain primary key, Referencing table contain foreign key.

* Referenced table / parent table :-

(a) Insert → No violence (In referencing table)
(b) Delete → may cause violence
↳ Solution → on delete set null.
on delete cascade.
on delete no action.
(c) Update → may cause violence
↳ Solution (i) Set null to referencing table on update
(ii) on delete cascade
(iii) on update no action.

* Referencing table / child table :-

(a) Insert → may cause violence in reference table
(b) Delete → No violence.
(c) update → may cause.

6) Composite key :-

→ Use more than one column.

* Mapping Cardinality / Degree of Relationship :-

(A) one to one :-

→ The relationship table has atleast two attributes, which are the primary keys of the two table.

→ Any attribute can be chosen as the primary key.

→ minimum two table.

Eid	Name	Age
E_1	N_1	22
E_2	N_2	20
E_3	N_3	18

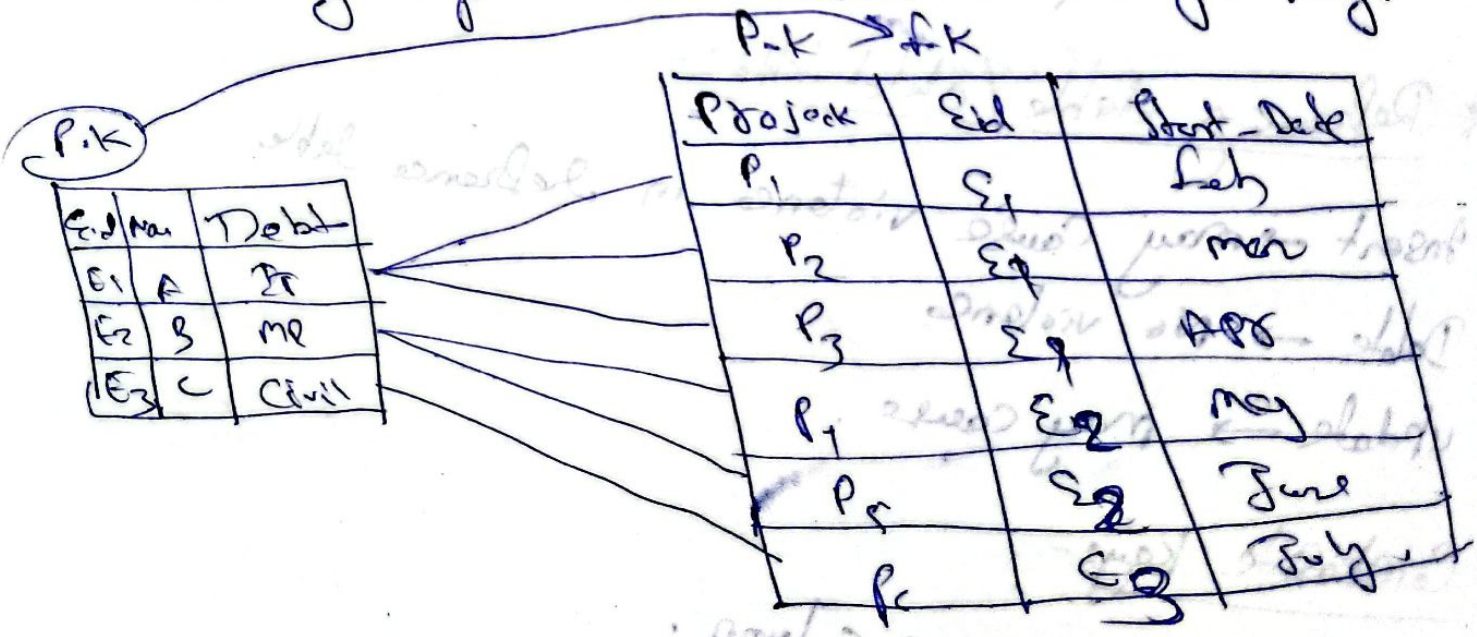
Eid	Did
E_1	D_1
E_2	D_2
E_3	D_3

Did	DName	Loc
D_1	DN_1	L_1
D_2	DN_2	L_2
D_3	DN_3	L_3

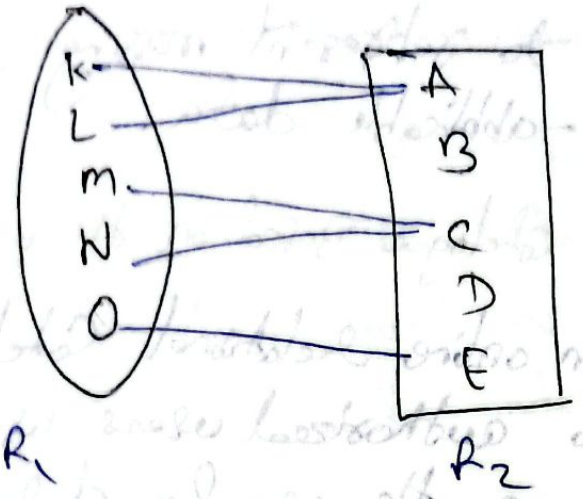
work

(B) one to many :-

→ In this type of relationship, the relationship table uses the primary keys of both tables as foreign key.



(c) Many to One :-



* Codd's Rule :- (total 12), (0 just for base).

(1) Rule Zero (Foundation / base rule)

→ For a system to be called an RDBMS, it must qualify as a relational database management system.
→ It must use its relational capabilities to qualify RDBMS

(2) Rule One (Information Rule)

→ All information in a database must be represented in one and only one way, as values in a table.
in the form of (row & column).

(3) Rule two - (Guaranteed access rule)

→ The entire data in a database must be accessible.
→ Every table must have a primary key.
→ Logically reachable using the primary key.

(4) Rule three - Systematic treatment of Null values.

→ Every field in an RDBMS must be allowed to be Null/Empty
→ Null values are used to represent missing data, unknown data or not-applicable data.

(5) Rule four :- Active online Catalogue Based on the Relational model

→ Each System must have an online relational Catalog
→ must be accessible to authorized users using the same regular Query language they use for data.

(6) Rule five - Data Sublanguage Rule.

→ must provide at least one relational language to access.
→ language should support linear syntax.
e.g. DDL, DML.

(7) Rule Six :- View updating rule.

→ View must be updatable.

(8) Rule Seven: Physical data Independence.

→ When any change is made in storage / disk, it must not affect the database table and their relationship

(9) Rule Seven :- High-level Insert, Delete and update rule.

→ must support insert, delete & update operations through a high-level relational language
→ Operation should work on a set of row (a retrieval set) not only on a single row at a time

(9) Rule Eight - Physical Data Independence.

→ when any change is made in storage/disk, it must not affect the database table or their relationship.

(10) Rule Nine - logical Data Independence.

→ when modifications are made at the logical level of the database, they must not affect the end user's view

→ Achieving logical data independence is difficult

(11) Rule ten :- Integrity Independence Rule.

→ According to this rule, Integrity Constraint must be stored in the database catalog, not inside application program.

Eg. In a student table, the constraint phone is NOT NULL is defined in the DB, so any inserted record must have a non-Null phone,

(12) Rule Eleventh :- Distribution Independence Rule.

→ The different parts of data that are distributed at various locations must be invisible to DB user.

(13) Rule Twelve :- Non-Subversion Rule.

→ Using any lower-level language you must not be able to ignore the integrity rule imposed by the DB language

→ constraints enforced by the DB lang. cannot be bypassed.

* Normalisation:-

→ Minimize data duplicacy/redundancy in a table.

→ When a table has data redundancy, the data can become redundancy, which leads to three types of anomalies/problem in the table:-

Sid	Name	Address	Cid	CName	teacher	fee
S_1	N_1	Jaipur	C_1	CN	T_1	Null
S_2	N_2	Jaipur	Null	C	T_2	10k
S_3	N_3	Kota	C_2	C++	T_3	15k
S_4	N_4	Udaipur	C_3	DS	T_4	null
S_4	N_5	Kota	Null	NULL	T_3	Null

(a) Insert Anomalies:-

→ When a record is inserted into a given table, for col like Cid, Name, teacher, fees, we sometimes have to enter Null or dummies value.

→ This increase the number of Null/dummy value in DB.

→ This situation called an insert anomaly.

(b) Delete anomaly:-

→ when a record is deleted from the DB, its data is also deleted.

(c) Update anomaly:-

→ When a column in the DB is updated, more than one related field has to be(or gets) updated.

Solution of Anamoly:-

- Decomposition of table

Types of Normalisation:

1 NF (Normal form) :-

→ According to RDBMS guidelines, every table should be in 1NF

→ Each attributes (column) of a table must hold only a single value on each row (no multivalued table)

R_No.	Name	DoB	C_Name
101	A	11/12/---	C,C++
10	B	17/11/---	Java
103	C	----	Py,C++,DS

→ C_Name had multiple values, so it is not in 1NF.

→ To convert it to 1 NF, three method can be used

(A) Repeat Roll no. (High degree dependency in 1NF)

(B) Repeat Column.

(C) Split table.

a: Split table:

Roll	Name	DoB
101	A	11/12
102	B	11/--
103	C	---

Roll no.	C_Name
101	C
101	C++
102	Java
103	Py
103	C++
103	DS

(Ques) Let a relation R(P,Q,R,S,T), whose candidate key are {P}, {P,R} and {R,S,T} is 1 NF or Not?

Ans Yes. 1 NF.

Reason

If some attribute is common to all candidate key then attribute is multivalued and the relation is not in 1 NF. Here, no attribute common to all three candidate key, so there is no multivalued attribute, hence the relation is in 1 NF

Q) Let a relation P(U,V,W,X,Y) in which candidate key. UV, VWX, VX, VY.

→ multivalued = Yes

→ 1 NF = No

Q) Let a relation R(P,Q,R,S,T) & candidate key are PR, PQ, RQ, ST, PST. How many multivalued attribute are there?

Ans. 0.

It is also 1 NF.

(2) Second Normal form :-

→ when a part of a candidate key (prime attribute) alone determine a non-prime attribute, it is called partial dependency.

→ must be in 1 NF.

→ No partial dependency.

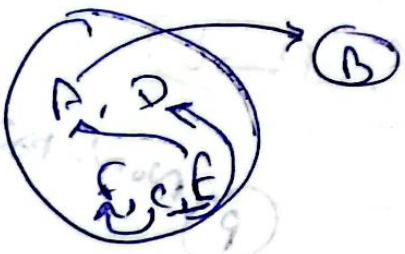
→ All non-prime attributes must be fully functionally dependent on candidate key.

Q.1) Relation $R_1 = (A, B, C, D, E, F)$

$FD = \{C \to F, E \to A, EC \to D, A \to B\}$

$C.K = \{A, D, FCE\}$.. (2NF?)

Ans.



No, it's not 2 NF.

because $A \to B$

↓ ↓

Prime Non Prime

Ques:- Relation $R_1 = (K, L, M, N, O, P)$

$FD = (MN \to P, P \to L, O \to N)$

$CK = (L, MN, O, P)$

Ans 2NF ✗



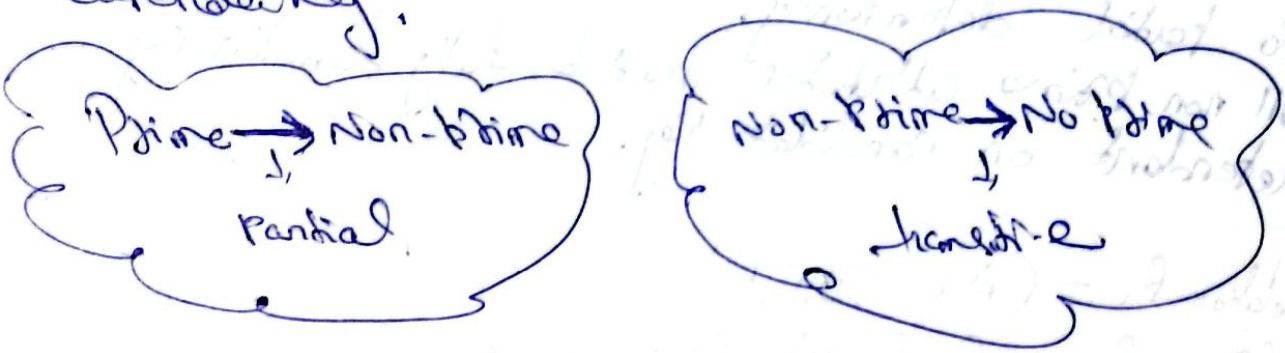
all within circle.

(3) Third Normal Form (3NF) :-

→ Must be 2NF.

→ Should be ~~also~~ No transitive dependency.

→ when one non-prime attribute determine another non-prime attribute, this is called transitive dependency.



(Q1) Relation $R_1 = (K, L, M, N, O, P)$

$FD = \{K \to M,\ MN \to O,\ O \to K,\ P \to L\}$

$C.K = \{K, M, ON\}$.





∴ No - 3NF.

(4) BCNF :-

→ Boyce Codd Normal form.

→ Relation 3NF.
- → atomic value
- → No partial dependence
- → No transitive dependency.

→ left hand side of each functional dependency is Superkey or Candidate key.

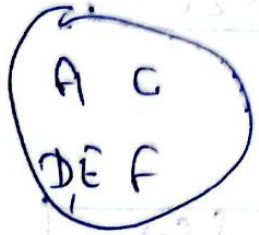
Q) Relation $R = (A, B, C, D, E, F)$

$CK = (A, C, D, F, E)$

$FD = \{ C \to D, F \to E, A \to E, D \to F \}$

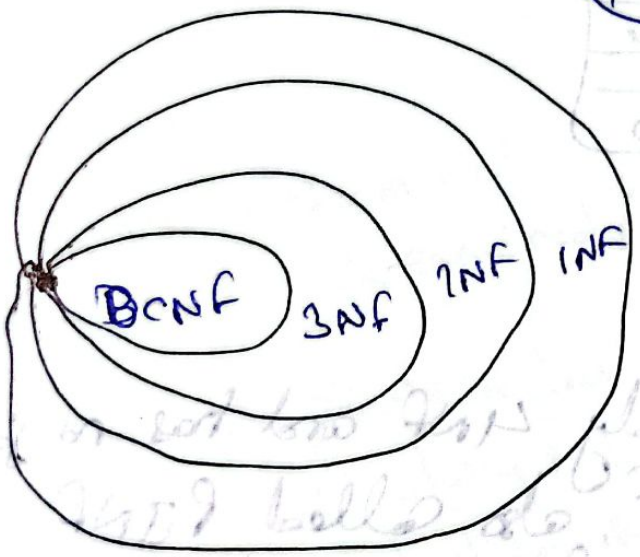
C.K C.K C.K C.K

Yes BCNF



all in circle.

Ex



5) 4th Normal form :-

→ 4th and 5th NF are highest level of normalisation

→ ~~Controls~~ 4NF controls multivalued dependency.

→ A multivalued dependency occurs when a table has more than one independent multivalued attribute.

→ For such a dependency we need at least three attributes: two independent attributes that both depended on a third attribute

→ Must be in BCNF.

→ No multivalued dependency.

Q.

SID	SName	CId	CName
S1	A	C1	C
S2	A	C2	D
S3	B	C1	C
S4	B	C2	D

multivalued dependency

$S_{ID} \twoheadrightarrow C_{ID}$

$S_{ID} \twoheadrightarrow C_{Name}$

$S_{Name} \twoheadrightarrow C_{Name}$

$R_1 \times R_2$

SID	SName
S1	A
S2	B

CID	CName
C1	C
C2	D

(6) 5th Normal form:-

→ Any table that is already 4NF and has no join dependencies is in 5NF also called PJNF (Project-Join Normal form)

Non-loss decomposition.

→ When we split table → to remove redundancy and later join/merge → to table back, no data should be lost.

* Functional Dependency.

→ The relationship between attributes of a relation is called functional dependency.

Sid	Name	Age.
101	N1	22
102	N2	25
103	N3	30
104	N4	27

→ From S_id we can uniquely identify Name.

So;

- S_id → Name.
- S_id is the determinant
- Name is the determined / dependent attribute.

for a determinant if its value is the same, the determined attribute's value will always be the same.
But if determined attribute has the same value, the determinant does not have to be the same.

* Types:

① Trivial f.D:-

→ if x, y are the attributes of relation R, then xy will be in trival f.D.

Y is subset of X.

$x \geq y$.

→ Every attribute determine itself.

S_id → S_id

$\underbrace{Sid + Name}_{x} = \underbrace{Name}_{y}$ } y is the subset of x

$x \geq y$.

→ The number of attributes (or set) on the left side includes all the attributes on the right side or

→ Both side have exactly the same attributes.

Q. Let Eid + Ename = Eid + E_Age. will trival FD?

A. Eid + Ename = Eid + Eage.

$x \cap y$

Eid + Ename = Eid + EAge.

Ename ≠ EAge.

$x \neq y$.

⇒ Not trival F.D.
but Semi trival F.D.

(B) Non-trival F.D :-

→ If x, y are the attributes of relation R, then it would be non-trival F.D.

if $x \cap y \neq 0$.

Sid ⇒ S_Name.

→ Non-trival FD, Determined they determine new attribute.

(c) Semi Trivial -FD:-

→ If x, y are the attribute of R, then then would be Semi trivial f.D if

Sid + Sname = Sid + S.Age. } y is not subset of x.

| SName - SAge |

* SQL Commands :-

(A) DDL (Data Definition language)

(a) Create (b) Alter (c) Drop (d) Truncate / Terminate

(B) DML (Data manipulation language)

(a) Select (b) Insert (c) Update (d) delete.

(C) DCL (Data Control language)

(a) Grant (b) Revoke.

(D) TCL (Transaction control language)

(a) Commit (b) Roll Back (c) Save point

(E) Constraints.

(a) Primary key Constraint

(b) Foreign "

(c) Unique "

(d) Check Constraint

(e) Default "

(f) Not Null "

* Working of Commands :-

(1) Create :-
- → Creation of database
- → Table name - Space not allowed use (_).

(2) Alter :-
- → Add new column in table
- → Drop/delete the column.
- → change data type
- → change Name/size
- → Table name change
- → Constraint add.
- → Table Structure update.

(3) Drop :-
- → used to delete both the table structure and its data together.
- → Deletes the complete table.
- → A table deleted using Drop cannot be recovered.

(4) Terminate/Truncate :-
- → This command deletes all data in the table at once.
- → It does not delete the table structure
- → The deleted data cannot be recovered.

(5) DELETE:-
→ Used to delete record (row) from a table.
→ we use WHERE clause so that only the selected data is deleted.
→ without a WHERE clause, all data in the table will be deleted.

(6) Insert:-
→ Used to insert new records into a table.
→ While inserting with this command, values must be given for all columns of the table.
→ NULL values can also be given.

(7) Update:-
→ To modify existing data in a table.
→ with update, if a WHERE clause is used, only the selected data is updated.
→ If no where all records in the table will be updated.

(8) Select:-
→ Standard Query by which we retrive table data

(9) GRANT:-
→ When we allocate some permission to user to the DB.

(10) Revoke:-
→ If want to deny the permission or deallocate the permission of the user of the DB.

(11) Commit:

→ The transaction/Queries done by the user are stored permanently in the DB.

(12) Roll back:

→ The last Committed transactions are revoked.

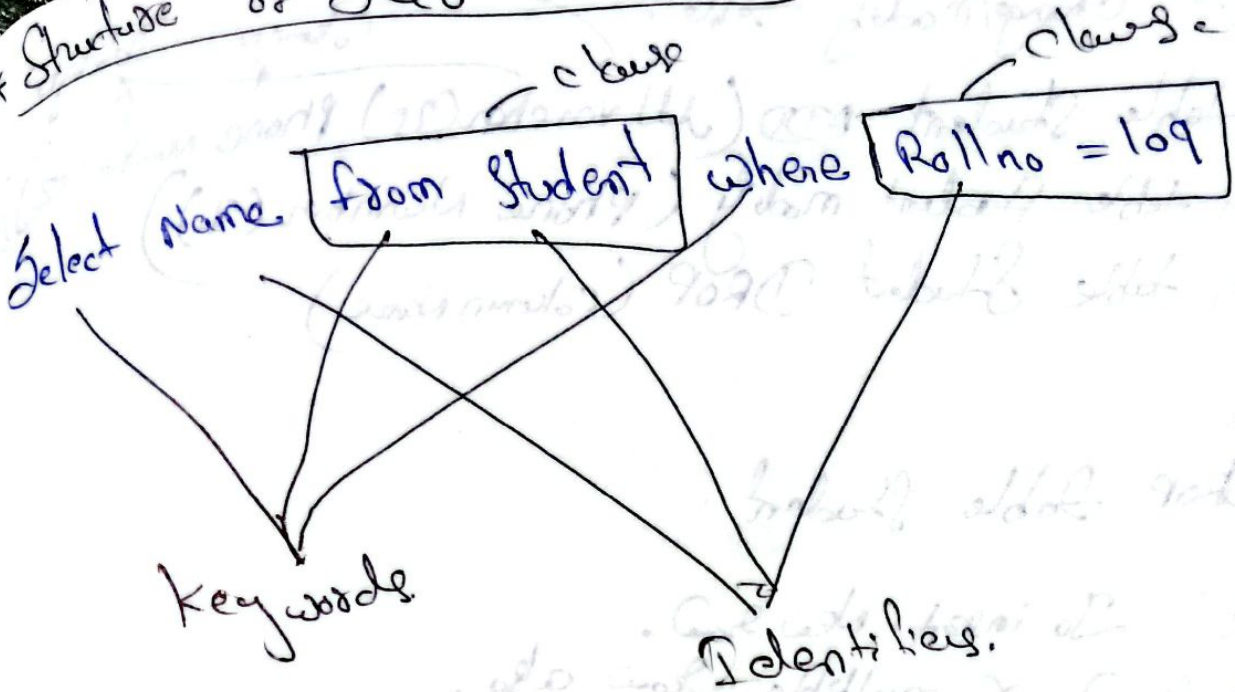
(13) Save point:-

→ Used to mark a particular point in transaction so that we can roll back only the selected part of the transaction.

* Data types:

1) Number → Numerical value, Number(n,d) (n: digit, d: decimal point)
2) char → max size 255
3) varchar → max size 2000 char.
4) Date → various Date format. DoB. Date.
5) long → Used to store variable length upto 2GB
6) Raw → 2000 Bytes. Store binary file.
7) long Raw → 2GB " " "
8) Time → Store time.

* Structure of SQL Command:-



* Sql logical operators:-

ALL, AND, ANY, BETWEEN, EXISTS, IN, LIKE, NOT, OR.

* Commands.

1) Create:- Create database Student;
use Student.

Create table tablename (CN-1, DT-1, CN-2 ..... CN-N-DT-N)

Create table Student (Regno Number(6), Name varchar2(25),
class Number(3), DoB Date, fee Number(6,3),
Primary key (Regno));

2) Describe:- (DESC) → Display all column with Constraints.

Describe Studen.

Name	Null?	Datatype
Regno	Not Null	Number(6)
Name		
class		
DoB		

(3) Alter:- change/modify table. (Column Name, Datatype)

→ Alter table Student ADD (Add varchar(25), Phone Number(12));

→ Alter table Student modify (Phone Number(7))

→ Alter table Student DROP (column Name)

(4) Drop:-

→ DROP Table Student.

(5) Insert: To insert New row.
Single row or multible row also.

Syntax: Insert into Student (Id, Name, Age, Phone) values
(103, 'Seeta', 19, 1234567891), (104, 'Ram', 21, 2468974456)

(6) Update :-

→ Table update (If no where condition)

→ Existing row update (If where cond')

→ Update Student Set Name = 'Malli' where ID = 103;

(7) DELETE :-

→ Delete existing row.

→ All record delete (If no where clause)

→ DELETE Student where ID = 101

(8) Select:

→ Retrieve data from DB.

→ Select * from table_Name.

Some Query:

by default ascending order

* Select ID, Name, Age from Student where class = 11 order by Name.

* Select age Count(*) As Number from Student Group by Age ; [Column virtually created for group by. Use As keyword]

* Select Age Sum(fee) As Amount from Student Group by Age having Sum(fee) > 20000;

* Select DISTINCT Name, Age, fees from Student order by Name. → Distinct apply on first. [It eliminate duplicate records].

* SQL functions :-

→ Result generation by manipulating available data items.
→ There are many built-in fn categorised in Group functi and Scalar functions.

i) Group function :-

→ function that act on a set of values.
→ A group function can take entire column of data as it's argument and produce a single value as result.

eg AVG(), COUNT(), SUM(), MIN(), MAX().

* SCALAR functions :-

→ Work on only one value.

Eg:-

Numeric function → ABS(), SQRT(), Power(), Round(),

String function → UPPER(), lower(), LENGTH(), LTRIM(), RTRIM(), SUBSTR()

Date function → LAST_DAY(), NEXT_DAY(), MONTH_BETWEEN()

Conversion function → TO_NUMBER, TO_CHAR, TO_DATE.

* SQL Constraints :-

→ These are the rules enforced on Data Columns on Table.

→ Ensure accuracy & reability of Data in table.

→ 2 Types :- (1) Column level (2) Table table.

(A) Not Null Constraint :-

→ By default Column can hold Null values

→ When a clum is defined not null then it become mandatory.

→ Eg Create table Product (PID Number(4) NotNull, PName varchar(25) Not Null, Company varchar(25), Price Number(6,3));

(B) Unique Key Constraint :-

→ This Constraint ensure that no rows have the same value in the specified Column.

→ Create table product (PID Number(4) Not Null unique, PName varchar(25)).

(c) Primary key Constraint :-

→ To identify record/row uniquely.
→ By default this column is Not Null & unique.
→ Create table product (PID Number(4) primary key,
PName varchar(25) Not Null);

3) Foreign key Constraint :-

→ used to provide relationship between two tables, that is why it is referred as reference key.
→ Foreign key is a primary key in another table.
→ Create table product (PID Number(4) Primary key,
PName varchar(25), Company varchar(25) REFRENCES Employee
(Company));

(e) Default Constraint :-

→ To set default value if no value pass through insert cmd.
→ Create table product (PID Number(4) primary key,
Price Number(6,3) DEFAULT 100000.000);

f) check Constraint :-

→ To apply TRUE/FALSE condition.
→ If the condition is true, then only we can insert the value in the Column.
→ Create table product (PID Number(3) Primary key,
PName varchar(25) CHECK (PName like 'P%'));

(2) Table Constraint :- v/s column constraint.

→ A constraint applied on a single column is called a column constraint

→ A constraint applied on a group of column called a Table constraint.

→ column constraint are written along with each column definition

→ Table constraint are written after all the column constraint are defined, at the end of the create table statement.

→ Create table product (PID Number(4) ← column constraint not null, Company varchar(25), PName varchar(25) not null, Primary key (PID, PName)) → table constraint.

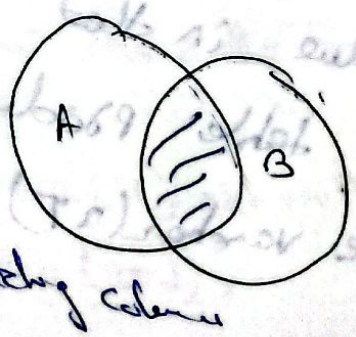
* SQL JOIN :-

→ To combine two or more table.

Types.

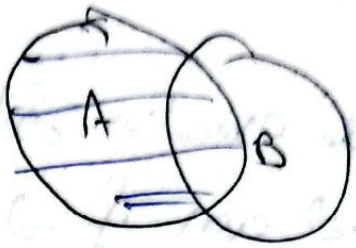
(1) Inner Join :-

```
Select table1.col1, table1.col2
from table1
INNER JOIN table2
ON table1.matchingcolumn = table2.matching column
```



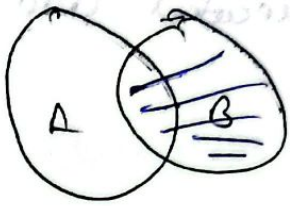
(b) Left Join :-

Select column_names from table1
left join table2 on table1.col
= table2.col;



(c) Right Join :-

→ Select column from table1
RIGHT JOIN table2 ON table1.col
= table2.col;



(d) full Join :-

Select column from table1 full join
table2 ON table1.coll = table2.col;



(e) Natural Join :-

→ # Connect two tables on the basis of their common column.
→ It return row by matching the values in those common column.
→ when inserting data into any data table, some defined rules are followed; these are called data Constrains

* Serialization :-

→ Describe the execution scheme of DB transaction.
→ It ensures that even if transactions are interleaved, their final result is the same as if the transactions were executed one at a time in some serial order.

Types..

(A) Conflict Serialization :-

→ When concurrency running transactions can be rearranged so their execution balance becomes equivalent to some serial order, the schedule is called conflict serializable

(B) View Serialization :-

→ When concurrently running transactions are already equivalent to a serial order and there is no need to rearrange them, ~~the schedule~~

* 2-phase locking :-

→ It is a concurrency control protocol that ensures consistency during the execution of database transactions.

Two phases :

(A) Growing phase :-

→ In this phase, the transaction acquires locks on all required resources
→ It can obtain as many locks as it needs in this phase, but cannot release any lock yet.

(b) Shrinking Phase:-
→ A transaction releases the locks that were allotted to it.
→ once a transaction enters this phase, it cannot obtain any new locks.

* Advantages of 2-phase locking:-
a) Deadlock Avoidance.
b) Ensuring Serializability. (Consistency of data in the DB is not violated)

* variation of two phase locking:-
a) Strict 2PL:- Release its exclusive locks only when it finishes.
b) Rigorous 2PL:-
→ A transaction can release none of its lock - neither shared nor exclusive - until it finishes.
c) Conservative 2PL:-
→ A transaction acquires all the locks it will need before starting execution, so the probably of deadlock is completely removed.

* Time-Stamp Concurrency Control:-
→ It is a transaction ordering method.
→ when multiple transactions are ready to execute, each is given timestamp (T), which is used to decide the order of the transaction.
→ The execution priority of each transaction is determined.

* No SQL:

→ not in SQL.
→ Non Relational Database.
→ Store the data - without Structure form.
→ Data stored by Scaling it.

Types.

(A) Horizontal Scaling:- Adding new server machines alongside the existing servers is called horizontal Scaling.

(B) vertical Scaling:- Adding more hardware/storage (RAM, hard disk etc). → on existing server machine is called vertical Scaling.

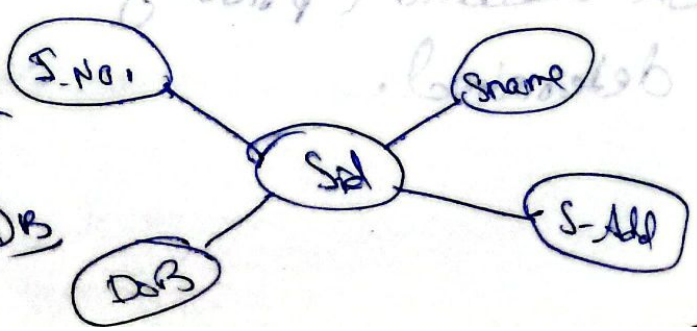
Types of NO-SQL DB:-

(A) Document based :- Data stored in JSON format.
Eg: MongoDB, CouchDB, Firebase.

(B) Key based :- Data stored in key-value pairs.
Eg: AmazonDB, RedisDB, Memcached.

(C) Wide/Column Based No SQL:- Database like RDBMS, Data stored in column instead of Row.
Eg: Apache Cassendra, Hbase, Google Bigtable.
→ use in Netflix, facebook .. etc.

(D) Graph-based :-
→ Data stored in form of Node & edges.
→ Amazon Neptune, StrongDB Neo4j.



Feature	Document based	key-value base	Column base	Graph based
Data model	JSON	key value pair	Column instead of row	Nodes & edges
Best use	Semi structured Data	Fast lookup & Caching	Analytics & Big Data	Relationship heavy data
Query performance	moderate	fast	High for analytical	Optimized for relationship
Schema	flexible	Dynamic	Semi structured	Schema less
Scalability	Horizontal	High Horizontal	High Scalable	Scale with relationship

* ACID Properties:

1) Atomicity :- Either all or none
→ Either Execute Completely or no part of it will execute.

2) Consistency :- Total amount before an transaction stays and after it ends must remain same.

A = 1000 B = 4000 — 5000
A = A - 500 B = B + 500
A = 500 B = 4500 — 5000

3) Isolation :- Parallel / Concurrent transaction should be executed in such a way that their effect is equivalent to some serial order. So that no inconsistency occur in the data.

4) Durability :-
→ once changes made by transaction are committed, they must become permanent in the database. So that even after any hardware or software failure. no committed data lost.